DOCUMENTO DE VISIÓN

Plantilla de Requisitos de Alto Nivel - Estándar RUP

Proyecto: ChuroViaje

Versión de Visión: 0.5 - Alineación con la arquitectura de Construcción

Fecha de Creación: 02/09/2026

Autor(es): Antonio Cuellar, Jorge Cabrera, Franz Caceres, Marcelo Quiroga y Alex Quispe

Organización: Universidad Católica Boliviana - SIS-227 Taller de Sistemas de Información / SIS-213 Ingeniería de Software

Control de Revisiones

Fecha	Versión	Descripción de Modificación	Autor
02/09/2026	0.1	Elaboración preliminar del Documento de Visión: planteamiento del problema, alcance, arquitectura y descripción de características.	Equipo ChuroViaje
11/09/2026	0.2	Cambio de nombre del proyecto a ChuroViaje; módulo de disponibilidad de surtidores confirmado en el alcance con fuente de datos vía grupos de WhatsApp (o API externa si se consigue); verificación de cuentas de usuario mediante correo Gmail de Google; actualización de docentes y materias asociadas.	Equipo ChuroViaje
25/09/2026	0.3	Se incorpora la decisión de arquitectura para el monitoreo de WhatsApp: servicio de extracción en Node.js (Baileys) y análisis de mensajes por lotes con un LLM. Se actualizan perspectiva del producto, suposiciones y dependencias, costos, restricciones, requisitos de sistema y glosario.	Equipo ChuroViaje
06/10/2026	0.4	Revisión de coherencia con los casos de uso, las historias de usuario y las decisiones tomadas para la Construcción: acceso solo con cuenta de Google, Mapbox como proveedor de mapas, WhatsApp como fuente principal de combustible con confirmación opcional de los conductores, telemetría solo con la app abierta, simulador que envía los datos al backend, estado de los surtidores por tipo de combustible y privacidad frente al proveedor del LLM. Se completan stakeholders, referencias y requisitos de documentación, y se actualiza la lista de autores.	Equipo ChuroViaje
13/10/2026	0.5	Alineación con el documento de arquitectura de Construcción: servidor en una instancia de Oracle Cloud sin costo mensual; red de calles del motor a partir de OpenStreetMap; motor core con núcleo en C++ y una cara en Java; backend con Spring Boot y PostgreSQL con PostGIS; análisis de mensajes con la capa gratuita de Gemini y números de teléfono ocultos; tipos de evento definidos y congestión detectada solo por el sistema; votos que retiran y confirman eventos; ruta alternativa que el conductor acepta o rechaza; excepciones de acceso del simulador y del servicio de monitoreo; puertos del servidor y plazos de conservación de los datos.	Equipo ChuroViaje

Tabla de Contenidos

> *Nota: La tabla de contenidos se actualizará automáticamente al abrir este archivo en Microsoft Word o Google Docs haciendo clic derecho sobre el campo y seleccionando 'Actualizar Campos'.*

1. Introducción

Esta introducción proporciona una visión general de todo el Documento de Visión del proyecto de software. Establece el propósito principal, alcance de la iniciativa, un glosario unificado de acrónimos y las referencias regulatorias o técnicas utilizadas.

1.1 Propósito

Este documento tiene como finalidad presentar a los docentes Ronal Mario Guerrero Guerrero, de la materia SIS-227 – Taller de Sistemas de Información, y Sandro Rodolfo Nieto Mendez, de la materia SIS-213 – Ingeniería de Software, de la Universidad Católica Boliviana, la especificación de alto nivel del sistema ChuroViaje, en cumplimiento del compromiso académico establecido para la evaluación final del semestre. Recolecta, analiza y define los requisitos funcionales de alto nivel del software, enfocándose en las necesidades del conductor tarijeño y en la justificación de por qué el producto debe existir.

1.2 Alcance

El sistema ChuroViaje automatizará el monitoreo colaborativo de tráfico vehicular y la optimización de rutas para conductores dentro de la mancha urbana de la ciudad de Tarija. A continuación se detalla qué características conforman el MVP (alcance incluido), qué reglas operativas las limitan, y qué queda explícitamente fuera de esta iteración del proyecto.

1.2.1 Lo que SÍ está incluido (El MVP)

Estas son las características innegociables que el equipo debe desarrollar y demostrar para aprobar la materia:

- **Reporte y visualización de eventos viales:** el sistema debe permitir a los usuarios marcar incidentes en el mapa e indicar hace cuánto ocurrieron. Los conductores reportan accidentes, controles policiales, bloqueos y otros eventos; la congestión no se reporta, la detecta el sistema. Una característica clave es la categorización por color (por ejemplo, rojo para accidentes, azul para controles). Además, los conductores pueden votar si un evento reportado por otro conductor es real o falso: con suficientes votos «falso» se retira del mapa y con suficientes votos «real» queda confirmado.
- **Motor de enrutamiento dinámico:** la aplicación debe calcular rutas desde un punto A hasta un punto B, permitiendo múltiples paradas. Crucialmente, cuando un evento vial afecta el camino que falta, debe proponer una ruta alternativa mejor, que el conductor acepta o rechaza.
- **Módulo de disponibilidad de combustible:** ChuroViaje debe consolidar y mostrar el estado de los surtidores por tipo de combustible.
- **Simulador de tráfico:** dado que es difícil tener miles de usuarios reales el día de la defensa del proyecto, es obligatorio desarrollar un script en Python que simule “usuarios fantasma” enviando sus datos al backend, para demostrar que el sistema funciona bajo carga.

1.2.2 Detalles Operativos (Reglas del Alcance Incluido)

Estas son las reglas técnicas y de negocio que limitan cómo funcionan las características incluidas en el MVP:

- **Límite geográfico estricto:** el sistema operará exclusivamente en la mancha urbana de Tarija. No se requiere que funcione en carreteras interprovinciales o en otras ciudades.
- **Navegación exclusivamente vehicular:** el motor de rutas trabaja con la red de calles de un extracto de OpenStreetMap filtrado a vías vehiculares, para evitar el enrutamiento peatonal (aceras). Mapbox solo se usa para mostrar el mapa en la app.
- **Telemetría solo con la app abierta:** la aplicación obtendrá la posición únicamente mientras está abierta en pantalla, con o sin una ruta activa, y nunca en segundo plano. Con esas posiciones el sistema calcula la velocidad, y si un vehículo permanece detenido o avanza muy lento en una vía vehicular durante un par de minutos, registra automáticamente una congestión.
- **Fuente de datos de surtidores:** el estado de los surtidores se obtiene del monitoreo de grupos de WhatsApp donde se comparte la disponibilidad de combustible, sin depender de reportes de los usuarios. Como complemento opcional, el conductor puede confirmar que acaba de recargar e indicar el tipo de combustible. Si en el futuro los surtidores dan acceso a una API externa, esta pasaría a ser la fuente primaria.
- **Sesión obligatoria:** todas las funciones de la app requieren haber iniciado sesión con una cuenta de Google; el sistema no maneja contraseñas propias. Las únicas excepciones son los usuarios fantasma del simulador, que entran con credenciales de prueba en un ambiente habilitado para eso, y el servicio de monitoreo de WhatsApp, que usa una credencial propia.

1.2.3 Lo que está FUERA de alcance (Exclusiones explícitas)

Esta es la sección más importante para proteger al equipo de desarrollo frente al docente evaluador. Estas características no se desarrollarán en esta iteración:

- **Cobertura nacional o regional:** cualquier zona fuera de la mancha urbana de Tarija.
- **Integración forzosa con gasolineras:** no es responsabilidad del equipo lograr una integración automatizada con los sistemas o la API de los surtidores. El sistema se apoya en el monitoreo de grupos de WhatsApp.
- **Rastreo en segundo plano:** la app no obtiene la ubicación del usuario cuando está minimizada o cerrada.
- **Monetización y Suscripciones:** el modelo de suscripción “Premium” (donde los usuarios pagan por ver el estado de los surtidores y tienen mayor “peso de confianza”) es una idea a futuro. No se debe programar, considerar ni presupuestar para la defensa de este proyecto.

1.3 Definiciones, Acrónimos y Abreviaciones

Esta sección define todos los términos de negocio, abreviaciones y acrónimos técnicos necesarios para interpretar el documento de visión sin ambigüedades.

Término / Acrónimo	Definición
RUP	Rational Unified Process. Metodología de desarrollo de software

	iterativa e incremental.
SUD	System Under Design. El sistema de software bajo diseño, tratado como una caja negra.
Crowdsourcing	Colaboración abierta distribuida: obtención de datos o servicios mediante los aportes de una comunidad de usuarios, en este caso los reportes viales de los conductores.
GPS	Global Positioning System. Sistema de posicionamiento global utilizado para obtener la ubicación del dispositivo del usuario, a partir de la cual el sistema calcula su velocidad.
API	Application Programming Interface. Interfaz que permite la comunicación entre la app, el backend y los servicios externos (mapas, Google Sign-In, proveedor del LLM).
REST	Representational State Transfer. Estilo de arquitectura utilizado por los endpoints del backend desarrollado en Java para la comunicación cliente-servidor.
VPS	Virtual Private Server. Servidor virtual privado. En este proyecto es una instancia de Oracle Cloud que aloja todos los componentes del servidor: backend, motor core, servicio de WhatsApp/LLM y base de datos.
OSM	OpenStreetMap. Proyecto colaborativo de mapeo libre; el motor de rutas usa un extracto de sus datos como red de calles, y Mapbox lo usa como base de su mapa.
SDK	Software Development Kit. Conjunto de herramientas oficiales (ej. Mapbox) utilizadas para integrar el mapa en la aplicación Flutter.
LLM	Large Language Model. Modelo de lenguaje de gran escala, ofrecido por un proveedor externo, que el sistema utiliza para interpretar por lotes los mensajes informales de los grupos de WhatsApp monitoreados. En este proyecto se usa la capa gratuita de la API de Gemini.
Baileys	Librería no oficial de código abierto para Node.js que permite conectarse a WhatsApp y leer los mensajes de los grupos de los que la cuenta es miembro.
Telemetría	Posiciones del dispositivo que la app envía al backend mientras está abierta en pantalla; con ellas el sistema calcula la velocidad y detecta congestión.
Wrapper	Componente en Java que es la cara del motor core: el backend lo usa como un objeto Java más, y el wrapper se comunica con el programa en C++ dentro del mismo servidor.

1.4 Referencias

Documentos del proyecto a los que hace referencia este Documento de Visión:

Título del Documento	Fecha	Organización / Fuente
Especificación de Casos de Uso de ChuroViaje (CU-01 a CU-08)	13/10/2026	Equipo ChuroViaje / Repositorio del proyecto
Historias de Usuario de ChuroViaje (HU-01 a	13/10/2026	Equipo ChuroViaje /

HU-13)		Repositorio del proyecto
Decisión de arquitectura: Conexión a WhatsApp para el monitoreo de surtidores (decision_conexion_whatsapp.md, v1.4)	12/10/2026	Equipo ChuroViaje / Repositorio del proyecto
Matriz de Requisitos y Trazabilidad de ChuroViaje	13/10/2026	Equipo ChuroViaje / SIS-213 Ingeniería de Software
Documento de Arquitectura de Software de ChuroViaje (03_des/02_Arquitectura)	13/10/2026	Equipo ChuroViaje / Repositorio del proyecto

2. Posicionamiento

2.1 Oportunidad de Negocio

La oportunidad de negocio surge al identificar que, al momento de necesitar llegar rápidamente a un destino, los conductores de la ciudad de Tarija dependen de aplicaciones como Google Maps, las cuales calculan la ruta más corta pero no consideran el tráfico real de la ciudad en tiempo real. Actualmente, solo los taxistas experimentados —quienes conocen empíricamente en qué calles y horarios se genera congestión— son capaces de tomar mejores decisiones de ruta que estas herramientas. Si bien existe una alternativa como Waze que sí incorpora reportes de tráfico colaborativos, su adopción en la ciudad es prácticamente nula, dejando un vacío de mercado local sin cubrir.

ChuroViaje busca aprovechar esta oportunidad ofreciendo, además del monitoreo colaborativo de tráfico, un diferenciador adicional que impulse su adopción: la disponibilidad de combustible en los surtidores en tiempo real, un dolor cotidiano del conductor que ninguna de las soluciones existentes resuelve.

2.2 Sentencia del Problema (Problem Statement)

Este marco estandariza la definición de los problemas clave que el proyecto resolverá:

El problema de...	*el congestionamiento vehicular en las vías principales de la ciudad de Tarija, que no cuenta con un sistema de monitoreo de tráfico en tiempo real, sumado a la incertidumbre sobre la disponibilidad de combustible en los surtidores locales.*
Afecta a...	*los conductores de la ciudad de Tarija, quienes se ven afectados por filas imprevistas en los surtidores y por la falta de información oportuna sobre el estado del tráfico.*
El impacto de lo cual es...	*pérdidas de tiempo en desplazamientos, obstrucción del tráfico regular por filas imprevistas en surtidores, y decisiones de ruta ineficientes al no existir una alternativa local económicamente viable a soluciones globales como Google Maps.*
Una solución exitosa sería...	*un sistema móvil colaborativo (crowdsourcing) que permita reportar y visualizar eventos viales en tiempo real, recalcular rutas dinámicamente evitando congestión, y consolidar la disponibilidad de combustible en los surtidores de la ciudad.*

2.3 Sentencia de Posicionamiento de Producto

Esta declaración consolida la propuesta de valor diferenciada del nuevo software en el mercado u organización:

Para...	*los conductores de la ciudad de Tarija.*
Quien...	*necesitan llegar más rápido a su destino evitando el tráfico y distintos eventos viales, además de conocer la disponibilidad de combustible en los surtidores antes de dirigirse a ellos.*
El (nombre de producto)...	*ChuroViaje*

Es un (tipo de producto)...	*una aplicación móvil multiplataforma (Android/iOS) de monitoreo de tráfico y optimización de rutas basada en crowdsourcing.*
Que...	*permite recalcular trayectorias en tiempo real a partir de los reportes de la propia comunidad de conductores y consolidar el estado de los surtidores de combustible de la ciudad.*
A diferencia de...	*soluciones globales como Google Maps, ChuroViaje no incurre en los costos de integración prohibitivos para un proyecto local y permite una personalización profunda de eventos cívicos y de infraestructura propios de la región de Tarija.*

3. Descripción de Stakeholders y Usuarios

Para proporcionar funciones que realmente añadan valor a la organización, se deben identificar a todos los involucrados (stakeholders) y los usuarios reales del sistema, mapeando sus perfiles y expectativas.

3.1 Resumen de Stakeholders

Los stakeholders son personas u organizaciones externas/internas que se ven afectadas por el desarrollo o que tienen poder de decisión:

Nombre del Rol	Descripción / Responsabilidad	Expectativa Clave
Docentes Evaluadores (SIS-227 y SIS-213)	Responsable de revisar el avance del proyecto y evaluar el cumplimiento del compromiso académico adquirido al inicio del semestre.	Verificar que el sistema cumpla con los objetivos planteados y que las funcionalidades comprometidas sean demostrables al final del semestre.
Conductores de la ciudad de Tarija	Usuarios finales de la aplicación. Aportan los reportes viales, los votos sobre los eventos y la confirmación de combustible.	Llegar más rápido a su destino y saber dónde hay combustible, con información confiable y sin que la app los rastree en segundo plano.
Equipo de desarrollo	Diseña, construye, despliega y mantiene el sistema. Cubre los roles de administrador del sistema, de infraestructura y de seguridad, y de equipo técnico que aparecen en los casos de uso.	Entregar las funcionalidades comprometidas dentro del semestre, con un sistema estable y demostrable bajo carga.
Administradores y miembros de los grupos de WhatsApp	Los administradores autorizan la presencia de la cuenta del proyecto en sus grupos; los miembros escriben los mensajes que el sistema interpreta.	Que no se almacenen sus datos personales y que el sistema no conserve el texto de sus mensajes. El proveedor del LLM sí lo recibe y puede usarlo (riesgo aceptado e informado en la política de privacidad).

3.2 Resumen de Usuarios

Los usuarios son quienes operarán directamente el software. Mapear sus perfiles garantiza una adecuada usabilidad y diseño de interacción:

Tipo de Usuario	Descripción de Actividades	Stakeholder Asociado
Conductor	Reporta y visualiza eventos viales categorizados por color, vota si son reales o falsos, consulta y sigue rutas optimizadas hacia su destino, consulta la disponibilidad de combustible en los surtidores de la ciudad y puede confirmar que acaba de recargar.	Conductores de la ciudad de Tarija
Desarrollador	Ejecuta el simulador de tráfico para probar el sistema bajo carga y revisa sus resultados.	Equipo de desarrollo

4. Descripción Global del Producto

4.1 Perspectiva del Producto

ChuroViaje es una aplicación independiente (stand-alone), no forma parte de un ERP existente. Se implementa bajo una arquitectura cliente-servidor distribuida compuesta por:

- Un frontend móvil multiplataforma desarrollado en Flutter (Android/iOS)
- Un backend de gestión desarrollado en Java con Spring Boot, con base de datos PostgreSQL y PostGIS, encargado de la autenticación de usuarios, los endpoints REST y la validación de los reportes de eventos
- Un motor core con un núcleo en C++, encargado de procesar los algoritmos sobre grafos (A*, con Dijkstra como alternativa) para la optimización de rutas y el análisis de telemetría. El backend lo usa a través de una cara en Java (wrapper) que se comunica con el programa en C++ dentro del mismo servidor, sin exponerlo a internet.
- Un servicio de WhatsApp/LLM desarrollado en Node.js, que lee los mensajes de los grupos de WhatsApp monitoreados mediante la librería Baileys, los interpreta por lotes con un LLM de un proveedor externo y envía el resultado al backend Java.
- Un simulador de tráfico desarrollado en Python, que genera usuarios fantasma para probar el sistema bajo carga.
- Como proveedor del mapa que ve el conductor se utiliza Mapbox (servicio externo con SDK maduro para Flutter). La red de calles con la que el motor calcula las rutas sale de un extracto de OpenStreetMap.
- Todos los componentes del servidor corren en una sola instancia de Oracle Cloud (Ubuntu sobre procesador ARM), dentro de su capa gratuita.

4.2 Resumen de Características

Mapeo de los requisitos funcionales de alto nivel (características) con su correspondiente valor o beneficio de negocio:

Característica Clave	Descripción Detallada	Beneficio de Negocio
Reporte y Visualización de Eventos Viales	Los usuarios reportan y visualizan en el mapa eventos categorizados por colores (ej. accidentes en rojo, controles en azul), indican hace cuánto ocurrieron y votan si los eventos de otros conductores son reales o falsos. La congestión la detecta el sistema a partir de la telemetría.	Brinda información de tráfico en tiempo real generada por la propia comunidad, sin depender de terceros.
Motor de Enrutamiento Dinámico	Calcula trayectorias entre un punto A y un punto B, considerando múltiples paradas y los eventos viales vigentes, y propone una ruta alternativa cuando un evento afecta el camino que falta.	Reduce el tiempo de viaje del conductor evitando el tráfico reportado.
Optimización del Consumo Energético	La app obtiene la posición mediante las APIs nativas del sistema operativo y solo mientras está abierta en pantalla; no usa el GPS en	Prolonga la duración de la batería del dispositivo del

	segundo plano. La velocidad la calcula el servidor.	usuario durante el uso de la aplicación.
Disponibilidad de Combustible	Consolida el estado de los surtidores por tipo de combustible (Gasolina, Diesel, Gasolina Premium o Diesel ULS; "Desconocido" cuando un mensaje no lo indica) y, cuando se conoce, el tamaño de fila, mediante el monitoreo de grupos de WhatsApp, complementado con la confirmación opcional de los conductores que acaban de recargar, quienes indican el tipo.	Evita filas imprevistas y pérdidas de tiempo al conductor.
Simulador de Tráfico	Script en Python que simula "usuarios fantasma" y envía al backend sus datos de ubicación y sus eventos, marcados como simulados para poder borrarlos sin tocar los datos reales.	Permite validar y demostrar el funcionamiento del sistema durante la defensa del proyecto.

4.3 Suposiciones y Dependencias

La disponibilidad continua de la instancia de Oracle Cloud que aloja los componentes del servidor, que Oracle mantenga su capa gratuita, la estabilidad del proveedor de mapas elegido (Mapbox), que los usuarios otorguen los permisos de ubicación necesarios mientras usan la app, que los grupos de WhatsApp monitoreados aporten información suficiente y confiable sobre el estado de los surtidores para sostener el módulo de disponibilidad de combustible, que el número dedicado del proyecto no sea bloqueado por usar un cliente no oficial de WhatsApp, la disponibilidad de la capa gratuita de la API de Gemini, con sus límites de uso, para interpretar los mensajes y la disponibilidad de Google Sign-In para el inicio de sesión.

4.4 Costo y Precio

La infraestructura del proyecto no tiene costo mensual de servidor: todos los componentes corren en una instancia de Oracle Cloud dentro de su capa gratuita. Se suma un dominio web (aprox. $15/año) con configuración DNS, necesario para exponer la API por HTTPS. El análisis de los mensajes de WhatsApp usa la capa gratuita de la API de Gemini, que no tiene costo pero sí límites de uso; analizar por lotes y solo cuando hay mensajes nuevos ayuda a no superarlos.

El modelo de sostenibilidad y monetización (esquema Freemium con una capa Premium de datos de surtidores en tiempo real) se plantea como un desarrollo próximo y no debe considerarse dentro del alcance ni del presupuesto actual del proyecto.

5. Restricciones

Restricciones operativas, tecnológicas y legales que limitan el diseño del sistema de información:

- Restricción de Plataforma: El frontend debe desarrollarse en Flutter para garantizar despliegue multiplataforma (Android/iOS), el backend de gestión debe desarrollarse en Java y el núcleo del motor core de enrutamiento en C++, con una cara en Java que lo comunica con el backend; el mapa de la app usa Mapbox. El servicio de WhatsApp/LLM se desarrolla en Node.js por depender de la librería Baileys, y el simulador en Python. Todo lo que corre en el servidor debe funcionar en un procesador ARM.
- Restricción de Idioma: La interfaz de usuario debe estar completamente en español.
- Restricción Legal: El sistema debe proteger los datos personales y de ubicación de los usuarios, evitando compartirlos con terceros sin su consentimiento; la ubicación solo se obtiene con la app abierta. Solo se monitorean grupos de WhatsApp cuyo administrador haya autorizado la presencia de la cuenta del proyecto; no se almacenan números ni nombres de los remitentes, y al proveedor del LLM se envía únicamente el texto y la fecha de los mensajes, con los números de teléfono ocultos. Como se usa la capa gratuita de Gemini, ese proveedor puede usar lo recibido para mejorar sus productos; ambas cosas deben indicarse en la política de privacidad del sistema.

6. Otros Requisitos del Producto (No Funcionales)

6.1 Requisitos de Sistema (Hardware/Software)

Una instancia de Oracle Cloud (Ubuntu, procesador ARM, 2 OCPU y 12 GB de memoria, en la capa gratuita) para alojar el backend (Java), el motor core (C++ con su cara en Java), el servicio de WhatsApp/LLM (Node.js) y la base de datos (PostgreSQL con PostGIS), dominio web con configuración DNS, dispositivos móviles Android o iOS con GPS y conexión a internet para ejecutar la aplicación Flutter, acceso al SDK de Mapbox como proveedor de mapas, un número telefónico dedicado para el monitoreo de WhatsApp y una credencial de la API de Gemini.

6.2 Requisitos de Desempeño (Rendimiento)

El sistema debe responder a las consultas de rutas en menos de 3 segundos bajo una carga concurrente típica de usuarios. Los eventos viales reportados por los usuarios deben reflejarse en el mapa de otros usuarios en un tiempo no mayor a 5-10 segundos; para lograrlo, la app consulta los eventos cada 5 segundos. El estado de un surtidor debe actualizarse en menos de 6 minutos desde que llega el mensaje de WhatsApp; para eso, mientras la lista de surtidores está abierta, la app la actualiza cada minuto. El consumo de batería adicional atribuible a la aplicación no debe superar los niveles habituales de una app de navegación estándar, ya que el GPS solo se usa con la app abierta.

6.3 Requisitos de Entorno y Seguridad

Seguridad de datos y comunicación:

- Todo el tráfico entre la app y el backend debe viajar cifrado por HTTPS/TLS. El motor C++ y la base de datos no se exponen a internet: el backend se comunica con ellos dentro del servidor.
- El sistema no almacena contraseñas: la identidad de cada usuario la verifica Google (Google Sign-In).
- Los datos de ubicación deben tratarse como sensibles: solo se obtienen con la app abierta, se minimiza su retención (el sistema no guarda las rutas ni las posiciones después de analizarlas, y borra los eventos y votos viejos al mes) y el simulador genera datos ficticios, marcados como simulados.

Autenticación y autorización:

- Autenticación basada en tokens JWT que duran una hora y se renuevan sin volver a pasar por Google durante 30 días, para las llamadas REST del backend Java.
- Acceso únicamente mediante inicio de sesión con cuenta de Google (Google Sign-In); todas las funciones de la app requieren sesión iniciada. Las excepciones son los usuarios fantasma del simulador, en un ambiente de pruebas, y el servicio de monitoreo de WhatsApp, con su propia credencial.

Integridad de los reportes colaborativos:

- Validación/rate-limiting para evitar que un usuario (o bot) sature el sistema con reportes falsos de eventos viales o con confirmaciones falsas de combustible.
- Los votos “real” o “falso” de los demás conductores permiten retirar del mapa los eventos falsos y confirmar los reales; el conductor no puede votar su propio evento ni la congestión que detecta el sistema.

Infraestructura (servidor en Oracle Cloud)

- Acceso SSH solo por llave (no contraseña). El firewall, configurado en Oracle Cloud y en el propio servidor, solo deja abiertos HTTPS, SSH y el puerto 80 para redirigir a HTTPS; la base de datos y el motor no se exponen.
- Mantener actualizado el sistema operativo y dependencias del servidor.

Cumplimiento y privacidad

- Política de privacidad clara indicando qué datos se recolectan (ubicación, reportes) y que no se comparten con terceros, con una única excepción: el texto y la fecha de los mensajes de los grupos de WhatsApp se envían al proveedor del LLM, sin datos del remitente y con los números de teléfono ocultos. La política debe aclarar que, en la capa gratuita de Gemini, Google puede usar ese texto para mejorar sus productos.
- Permisos de ubicación explícitos y revocables por el usuario desde la app.

7. Requisitos de Documentación y Empaque

7.1 Manuales y Ayuda

El equipo entregará un manual de usuario para el conductor, con las funciones principales de la app (reportar y ver eventos, calcular rutas y consultar combustible), y una guía de instalación y despliegue del backend, el motor core, el servicio de WhatsApp/LLM y el simulador.

7.2 Etiquetado y Empaque

La aplicación y sus manuales mostrarán el nombre y el logotipo de ChuroViaje. Los documentos académicos llevan además la identificación de la Universidad Católica Boliviana. El mapa debe mostrar la atribución que exigen Mapbox y OpenStreetMap.